

# **TJSC 2026 - Analista de Sistemas**

## **Engenharia de Software e Desenvolvimento**

### **Aula 7 - Orientação a objetos: conceitos fundamentais, análise e modelagem**

Material autoral orientado ao edital retificado do TJSC 2026 e calibrado ao padrão de cobrança da FGV. A aula cobre orientação a objetos sem avançar indevidamente para padrões de projeto, que ficam para aula própria.

|                |                                                                          |
|----------------|--------------------------------------------------------------------------|
| Disciplina     | Engenharia de Software e Desenvolvimento                                 |
| Aula           | 7 de 17                                                                  |
| Item do edital | 1.8 Orientação a objetos: conceitos fundamentais, análise e modelagem    |
| Banca          | FGV - Fundação Getulio Vargas                                            |
| Foco           | Conceitos formais, modelagem OO, leitura de código e pegadinhas de prova |

# 1. Escopo exato da aula

Esta aula cobre a Aula 7 da trilha atualizada: Orientação a objetos: conceitos fundamentais, análise e modelagem. O recorte corresponde ao item 1.8 do edital da disciplina Engenharia de Software e Desenvolvimento. O edital também prevê padrões de projeto e UML 2.1, mas eles aparecem em aulas próprias. Aqui, quando necessário, a UML - Unified Modeling Language - será citada apenas como linguagem de modelagem para representar classes, relacionamentos e responsabilidades, sem aprofundar diagramas específicos.

O objetivo é transformar sua experiência prática com Java, Quarkus, Angular, TypeScript, APIs e sistemas corporativos em acerto de prova. A FGV costuma punir o candidato que programa bem, mas confunde definição formal: objeto com classe, herança com composição, sobrescrita com sobrecarga, encapsulamento com simples uso de getters e setters, interface com classe abstrata, e análise orientada a objetos com desenho físico de banco ou implementação de framework.

| Bloco                       | O que será coberto                                                                                            | Como pode cair                                |
|-----------------------------|---------------------------------------------------------------------------------------------------------------|-----------------------------------------------|
| Conceitos fundamentais      | Objeto, classe, instância, atributo, método, mensagem, estado, comportamento e identidade                     | Definição conceitual, V/F e leitura de código |
| Pilares de OO               | Encapsulamento, abstração, herança e polimorfismo                                                             | Alternativas próximas e exemplos Java         |
| Relações                    | Associação, dependência, agregação, composição, generalização, realização e multiplicidade                    | Cenário de modelagem e relação entre classes  |
| Análise orientada a objetos | Identificação de classes, responsabilidades, colaborações, fronteira-controlador-entidade e modelo de domínio | Caso prático com requisitos                   |
| Modelagem OO                | Modelo conceitual vs modelo de projeto vs código; visibilidade; coesão; acoplamento                           | Diferença entre uso real e cobrança FGV       |

## 2. Como a FGV vem cobrando orientação a objetos

Nas provas recentes de TI da FGV, orientação a objetos aparece com forte carga contextual: um analista modela uma hierarquia de funcionários, revisa código Java, identifica **classe abstrata**, avalia sobrescrita de método, associa encapsulamento a alta coesão ou identifica candidatos a classes na análise orientada a objetos. O enunciado tende a trazer um mini-caso, não apenas uma pergunta de dicionário.

O padrão a ser simulado nesta aula é: cenário institucional, nomes de classes e métodos, alternativas tecnicamente próximas e uma resposta defensável. A FGV também gosta de misturar termos verdadeiros em contexto errado. Exemplo: 'herança' é conceito real, mas não explica todo reaproveitamento; 'interface' é contrato, mas não é automaticamente herança múltipla de implementação; '@Override' indica sobrescrita, mas não torna o método abstrato.

Observação de foco: SOLID - Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation e Dependency Inversion - e padrões de projeto serão aprofundados em outra aula. Aqui só aparecerão se ajudarem a distinguir conceitos fundamentais de OO.

## 3. Orientação a objetos: ideia central

Orientação a objetos é um paradigma de análise, projeto e programação que organiza o sistema em torno de objetos que possuem estado, comportamento e identidade. Em vez de pensar apenas em procedimentos que manipulam dados soltos, o modelo OO busca representar entidades do domínio e responsabilidades de software por meio de objetos que colaboram entre si.

Em Java, um objeto pode representar um DocumentoProcesso, uma Solicitação, um Usuário, uma OrdemPagamento ou uma AssinaturaEletrônica. O objeto reúne dados relevantes e operações

relacionadas. Em uma API Quarkus, isso pode aparecer como entidade de domínio, classe de aplicação, repositório, DTO - Data Transfer Object - ou controller, mas a prova cobra principalmente a definição formal e a coerência da modelagem.

| Termo           | Definição de prova                                                         | Exemplo em sistema corporativo                   |
|-----------------|----------------------------------------------------------------------------|--------------------------------------------------|
| Objeto          | Entidade de software com estado, comportamento e identidade                | pedido123 com status, valor e operação aprovar() |
| Classe          | Molde, tipo ou especificação a partir do qual objetos são instanciados     | classe Pedido                                    |
| Instância       | Objeto concreto criado a partir de uma classe                              | new Pedido(...)                                  |
| Atributo        | Dado ou propriedade que representa parte do estado do objeto               | status, valorTotal, dataCriacao                  |
| Método/operação | Comportamento que o objeto pode executar ou serviço oferecido pela classe  | aprovar(), cancelar(), calcularTotal()           |
| Mensagem        | Solicitação enviada a um objeto para que execute uma operação              | pedido.aprovar(usuario)                          |
| Identidade      | Característica que diferencia um objeto de outro, mesmo com estados iguais | duas petições com mesmo texto mas ids distintos  |

### 3.1 Classe não é objeto

Classe é a definição. Objeto é a ocorrência concreta em tempo de execução. A classe ProcessoJudicial descreve quais atributos e métodos os objetos desse tipo terão. Um processo específico, com número, classe processual e situação próprios, é objeto/instância. Em prova, alternativa que trata classe como 'registro concreto' ou objeto como 'molde' inverte os conceitos.

```
public class ProcessoJudicial {
    private String numero;
    private String situacao;

    public void movimentar(String novaSituacao) {
        this.situacao = novaSituacao;
    }
}

ProcessoJudicial p1 = new ProcessoJudicial(); // objeto/instância
ProcessoJudicial p2 = new ProcessoJudicial(); // outro objeto/instância
```

### 3.2 Estado, comportamento e identidade

O estado é formado pelos valores dos atributos em certo momento. O comportamento é o conjunto de operações que alteram, consultam ou usam esse estado. A identidade separa objetos distintos mesmo quando seus atributos parecem iguais. Essa distinção é útil para a FGV porque ela permite diferenciar objeto de valor, instância de classe e igualdade de identidade.

| Elemento      | Pergunta mental                                   | Pegadinha                                                                        |
|---------------|---------------------------------------------------|----------------------------------------------------------------------------------|
| Estado        | Como o objeto está agora?                         | Confundir estado com classe. A classe define atributos; o objeto tem valores.    |
| Comportamento | O que o objeto sabe fazer?                        | Colocar regra de negócio fora do objeto e dizer que ainda é alto encapsulamento. |
| Identidade    | Este objeto é o mesmo ou apenas tem dados iguais? | Confundir igualdade lógica com mesma referência/mesma ocorrência.                |

## 4. Encapsulamento

Encapsulamento é o princípio de agrupar dados e comportamentos relacionados dentro de uma unidade, controlando o acesso ao estado interno. Ele não é apenas 'colocar atributo private'. Em prova, a melhor leitura é: **o objeto protege seu estado e oferece operações que preservam invariantes e regras de negócio.**

**Invariante** é **uma regra que deve permanecer verdadeira para o objeto estar consistente**. Exemplo: uma Petição não pode ser protocolada sem parte autora; uma OrdemPagamento não pode ser aprovada com valor negativo; um DocumentoProcesso não deve aceitar classificação menor que 1. Encapsular bem é impedir que qualquer trecho externo deixe o objeto inválido.

```
public class OrdemPagamento {
    private BigDecimal valor;
    private StatusPagamento status;

    public OrdemPagamento(BigDecimal valor) {
        if (valor == null || valor.signum() <= 0) {
            throw new IllegalArgumentException("Valor deve ser positivo");
        }
        this.valor = valor;
        this.status = StatusPagamento.PENDENTE;
    }

    public void aprovar(Usuario usuario) {
        if (!usuario.podeAprovarPagamento()) {
            throw new RegraNegocioException("Usuário sem alçada");
        }
        this.status = StatusPagamento.APROVADO;
    }
}
```

No código acima, o atributo valor é privado e só pode ser definido por caminho validado. A operação aprovar() concentra comportamento e regra de negócio. Isso é muito mais forte, para prova, do que uma classe anêmica com atributos privados e setters públicos irrestritos.

### 4.1 Modificadores de acesso e encapsulamento em Java

| Modificador     | Acesso                          | Como pode cair                                                    |
|-----------------|---------------------------------|-------------------------------------------------------------------|
| private         | Apenas dentro da própria classe | Subclasse não acessa diretamente atributo private da superclasse. |
| sem modificador | Pacote/package                  | Acesso por classes do mesmo pacote.                               |
| protected       | Pacote e subclasses             | Não significa 'público para todos'.                               |
| public          | Qualquer classe acessível       | Aumenta acoplamento se usado sem critério.                        |

## 5. Abstração

Abstração é selecionar os aspectos essenciais de um objeto ou conceito e omitir detalhes irrelevantes para determinado propósito. Em análise orientada a objetos, abstrair é decidir quais classes, atributos, operações e relacionamentos representam adequadamente o domínio do problema. Em programação, classes abstratas e interfaces são mecanismos para expressar abstrações, mas abstração não se resume a esses mecanismos.

Exemplo: para um sistema de tramitação, DocumentoProcesso pode ter número, tipo, classificação e operação assinar(). O detalhe de como o arquivo é armazenado em disco, em object storage ou em banco pode ser irrelevante no modelo de domínio inicial. A abstração correta depende do objetivo do modelo.

|                |                   |
|----------------|-------------------|
| Confusão comum | Correção de prova |
|----------------|-------------------|

|                                            |                                                                                                                                                       |
|--------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| Abstração = esconder dados                 | Esconder detalhes de implementação é mais próximo de encapsulamento; abstração é focar no essencial.                                                  |
| Classe abstrata = qualquer classe genérica | Classe abstrata é um mecanismo que impede instanciação direta e pode conter métodos abstratos e concretos.                                            |
| Interface = classe abstrata sem atributos  | Interface é contrato de comportamento; em Java moderno pode ter métodos default/static, mas não representa estado de instância como uma classe comum. |

## 6. Herança, generalização e especialização

Herança é um mecanismo pelo qual uma classe derivada herda características de uma classe base. Na modelagem, a relação conceitual é frequentemente descrita como **generalização/especialização**: a superclasse generaliza características comuns, e as subclasses especializam comportamento ou dados.

A regra mental é 'é um tipo de' (is-a). ServidorEfetivo é um tipo de Servidor. PeticaoInicial é um tipo de DocumentoProcesso. Já DocumentoProcesso possui ArquivoDigital; isso é relação de composição/associação, não herança.

| Relação               | Pergunta                  | Exemplo correto                 | Erro clássico                                                |
|-----------------------|---------------------------|---------------------------------|--------------------------------------------------------------|
| Herança/generalização | X é um tipo de Y?         | Mandado é um DocumentoProcesso  | Usar herança para reaproveitar código sem relação conceitual |
| Composição            | X é parte essencial de Y? | Processo possui Movimentações   | Dizer que Movimento herda de Processo                        |
| Associação            | X se relaciona com Y?     | Servidor está lotado em Unidade | Confundir qualquer FK com herança                            |

### 6.1 Herança em Java: ponto de prova

Em Java, uma classe pode estender diretamente apenas uma superclasse, mas pode implementar múltiplas interfaces. Toda classe, direta ou indiretamente, deriva de Object, mas isso não deve ser confundido com herança múltipla real de classes. A FGV já explorou a ideia de que interfaces fornecem suporte a múltiplos comportamentos sem as desvantagens da herança múltipla de implementação.

```
abstract class DocumentoProcesso {
    private String numero;

    public DocumentoProcesso(String numero) {
        this.numero = numero;
    }

    public String getNumero() {
        return numero;
    }

    public abstract void validar();
}

class Peticao extends DocumentoProcesso {
    public Peticao(String numero) {
        super(numero);
    }

    @Override
    public void validar() {
        // regras específicas de petição
    }
}
```

A classe abstrata DocumentoProcesso impede instanciação direta e exige que subclasses implementem validar(). A subclasse Peticao herda estado e comportamento acessível, mas não acessa diretamente atributos private da superclasse.

## 7. Polimorfismo

Polimorfismo significa 'muitas formas'. Em orientação a objetos, é a capacidade de objetos de diferentes classes responderem à mesma mensagem/operação de formas específicas. A FGV costuma diferenciar polimorfismo por sobrescrita, sobrecarga e uso de interface.

| Tipo de situação           | O que ocorre                                                        | Momento típico                  | Exemplo                                                |
|----------------------------|---------------------------------------------------------------------|---------------------------------|--------------------------------------------------------|
| Sobrescrita/overriding     | Subclasse redefine método herdado com assinatura compatível         | Execução, por despacho dinâmico | Peticao.validar() redefine DocumentoProcesso.validar() |
| Sobrecarga/overloading     | Mesma classe possui métodos com mesmo nome e parâmetros diferentes  | Compilação, pela assinatura     | buscar(id), buscar(numero, ano)                        |
| Polimorfismo por interface | Variável do tipo interface referencia objetos de classes diferentes | Execução                        | Assinavel doc = new Peticao()                          |

```
interface Assinavel {
    void assinar(Usuario usuario);
}

class Peticao implements Assinavel {
    public void assinar(Usuario usuario) { /* assinatura de petição */ }
}

class Sentenca implements Assinavel {
    public void assinar(Usuario usuario) { /* assinatura de sentença */ }
}

List<Assinavel> documentos = List.of(new Peticao(), new Sentenca());
for (Assinavel d : documentos) {
    d.assinar(magistrado); // mesma mensagem, comportamento específico
}
```

### 7.1 Binding dinâmico e método de instância

Ligação dinâmica, ou late binding, é a **escolha da implementação do método em tempo de execução com base no tipo real do objeto**, e não apenas no tipo declarado da variável. Em Java, métodos de instância sobrescritos são despachados dinamicamente. Métodos static são resolvidos de outra forma e não representam polimorfismo por sobrescrita no mesmo sentido.

Pegadinha: @Override não significa que o método é abstrato. A anotação indica que o método sobrescreve ou implementa método declarado em supertipo. A abstração está na declaração abstract ou no contrato da interface, não na anotação em si.

## 8. Classe abstrata e interface

Classe abstrata e interface servem para expressar abstrações, mas com funções distintas. **Classe abstrata é útil quando há uma hierarquia conceitual com estado e comportamento comum. Interface é útil para definir capacidade, contrato ou papel que várias classes podem cumprir, mesmo sem compartilhar uma superclasse comum.**

| Critério      | Classe abstrata                      | Interface                                       |
|---------------|--------------------------------------|-------------------------------------------------|
| Ideia central | Base parcial de uma hierarquia       | Contrato de comportamento                       |
| Instanciação  | Não pode ser instanciada diretamente | Não é instanciada diretamente como objeto comum |

|         |                                         |                                                                |
|---------|-----------------------------------------|----------------------------------------------------------------|
| Estado  | Pode ter atributos de instância         | Não modela estado de instância como classe comum               |
| Métodos | Pode ter métodos concretos e abstratos  | Define comportamentos; em Java moderno pode ter default/static |
| Herança | Uma classe estende no máximo uma classe | Uma classe pode implementar múltiplas interfaces               |
| Exemplo | DocumentoProcesso                       | Assinavel, Auditavel, Exportavel                               |

## 9. Relações entre classes e objetos

Em análise e modelagem OO, não basta identificar classes. É preciso representar como elas se relacionam. A FGV pode cobrar a diferença entre associação, dependência, agregação, composição, generalização e realização, principalmente por meio de caso prático.

| Relação       | Significado                                             | Exemplo                                                 | Erro típico                                    |
|---------------|---------------------------------------------------------|---------------------------------------------------------|------------------------------------------------|
| Associação    | Vínculo estrutural entre classes                        | Processo possui Partes; Servidor está lotado em Unidade | Confundir com herança                          |
| Dependência   | Uma classe usa outra de forma pontual                   | ServicoRelatorio usa GeradorPDF                         | Modelar dependência fraca como composição      |
| Agregação     | Relação todo-parte fraca; parte pode existir sem o todo | Equipe agrega Servidores                                | Tratar como ciclo de vida obrigatório          |
| Composição    | Relação todo-parte forte; parte depende do todo         | Pedido composto por ItensPedido                         | Dizer que todo relacionamento 1:N é composição |
| Generalização | Relação de herança; subtipo é um tipo do supertipo      | Peticao é Documento                                     | Usar para 'tem um'                             |
| Realização    | Classe cumpre contrato de interface                     | Peticao realiza Assinavel                               | Confundir com herança de implementação         |

A multiplicidade indica quantas instâncias podem participar da relação: 1, 0..1, 0..\*, 1..\*. Em OO, multiplicidade ajuda a decidir se um atributo será uma referência simples, uma coleção, uma dependência de método ou uma associação persistida.

## 10. Análise orientada a objetos - AOO

AOO - Análise Orientada a Objetos - procura **compreender o domínio do problema e identificar objetos, classes, responsabilidades e colaborações relevantes antes de decidir detalhes técnicos de implementação**. O foco é o que o sistema deve representar e como os conceitos do domínio colaboram para atender aos requisitos.

### 10.1 Do requisito para classes candidatas

**Uma técnica clássica é procurar substantivos e expressões nominais nos requisitos**, mas isso não deve ser aplicado mecanicamente. Nem todo substantivo vira classe; nem todo verbo vira método; nem todo ator externo vira entidade persistida. A decisão depende de identidade, ciclo de vida, responsabilidades e relevância para o sistema.

|                     |                       |         |
|---------------------|-----------------------|---------|
| Pergunta de análise | Se a resposta for sim | Exemplo |
|---------------------|-----------------------|---------|

|                                                |                                                                    |                                                   |
|------------------------------------------------|--------------------------------------------------------------------|---------------------------------------------------|
| O conceito tem identidade própria?             | Pode ser classe de domínio                                         | Processo, Documento, Parte                        |
| O conceito tem ciclo de vida relevante?        | Tende a ser classe                                                 | Assinatura Eletrônica criada, validada, revogada  |
| O conceito possui regras próprias?             | Pode concentrar comportamento                                      | OrdemPagamento.aprovar()                          |
| É apenas uma característica de outro conceito? | Pode ser atributo                                                  | dataProtocolo em Petição                          |
| É uma ação ou evento?                          | Pode ser método, classe de evento ou entidade, conforme relevância | Publicação pode ser evento ou entidade Publicacao |

## 10.2 Classes de entidade, fronteira e controle

Em muitos materiais de análise OO, aparece a separação entre **classes de entidade, fronteira e controle, útil para entender responsabilidades**. Entidade representa informação e comportamento do domínio; fronteira interage com usuário ou sistema externo; controle coordena um caso de uso ou fluxo de aplicação.

| Tipo               | Papel                                                | Exemplo em Java/Quarkus/Angular                     | Pegadinha                                     |
|--------------------|------------------------------------------------------|-----------------------------------------------------|-----------------------------------------------|
| Entidade           | Representa conceito do domínio com estado e regras   | ProcessoJudicial, DocumentoProcesso, OrdemPagamento | Não confundir automaticamente com tabela JPA  |
| Fronteira/boundary | Faz interface com ator, tela, API ou sistema externo | REST resource, componente Angular, client HTTP      | Não deve concentrar regra de domínio complexa |
| Controle/control   | Coordena fluxo de caso de uso                        | ServicoProtocolarDocumento, ApplicationService      | Não é entidade de domínio só por ter métodos  |

## 10.3 Modelo de domínio, modelo de projeto e código

Modelo de domínio **representa conceitos do negócio**. Modelo de projeto **refina decisões de software, camadas, interfaces, visibilidade e padrões arquiteturais**. Código é a implementação concreta. Em um sistema real, uma classe Java pode misturar detalhes de persistência, serialização e validação; para a prova, é essencial separar conceito de implementação.

| Nível           | Foco                                       | Exemplo                                                  |
|-----------------|--------------------------------------------|----------------------------------------------------------|
| Análise/domínio | Conceitos e regras do problema             | Processo, Parte, Documento, Assinatura                   |
| Projeto/design  | Responsabilidades técnicas e colaborações  | DocumentoService, AssinaturaClient, RepositorioDocumento |
| Implementação   | Código, framework, anotações, persistência | @Entity, @RestController, @Inject, repository Panache    |

# 11. Coesão e acoplamento na modelagem OO



Coesão mede o quanto as responsabilidades de uma classe estão relacionadas entre si. Alta coesão é desejável: a classe tem propósito claro. Acoplamento mede o grau de dependência entre classes ou módulos. Baixo acoplamento é desejável: mudanças em uma classe têm menor impacto em outras.

Para a FGV, a pegadinha é trocar os dois: encapsulamento e alta coesão aparecem quando a classe protege seu estado e concentra comportamentos coerentes; baixo acoplamento aparece quando uma classe conhece menos detalhes internos de outra e depende de abstrações ou contratos, quando cabível.

| Situação                                                                                                    | Diagnóstico                            |
|-------------------------------------------------------------------------------------------------------------|----------------------------------------|
| Classe ProtocoloController valida regra de negócio, envia e-mail, acessa banco, gera PDF e chama mensageria | Baixa coesão; muitas responsabilidades |
| Classe ProcessoJudicial controla transições válidas de situação processual                                  | Alta coesão no domínio                 |
| ServicoAcessaSistemaB instância diretamente implementações concretas e conhece detalhes de protocolo        | Alto acoplamento                       |
| Serviço depende de interface GatewayAssinatura e recebe implementação por injeção                           | Menor acoplamento                      |

## 12. Diferença entre prática real e prova

No trabalho, você vê classes DTO, entities JPA, services, controllers, mappers, clients REST, repositories e componentes Angular. A prova pode usar nomes semelhantes, mas quer a **definição conceitual**. Um **DTO não é classe de domínio necessariamente; uma entidade JPA não garante bom modelo OO**; um service procedural com métodos estáticos não vira OO apenas porque está em uma classe.

| No trabalho                   | Na cobrança FGV                                                                                                        |
|-------------------------------|------------------------------------------------------------------------------------------------------------------------|
| Classe Java com @Entity       | Pode representar entidade persistida, mas a questão pode estar perguntando classe de domínio ou classe OO, não tabela. |
| DTO usado para trafegar JSON  | É objeto/classe, mas normalmente não concentra regra de negócio do domínio.                                            |
| Service em Quarkus            | Geralmente coordena aplicação; não confundir com objeto de domínio rico.                                               |
| Interface para client externo | Contrato; pode viabilizar polimorfismo e baixo acoplamento.                                                            |
| Componente Angular            | Classe de front-end; pode participar de boundary/fronreira, não de domínio do back-end.                                |

## 13. Pegadinhas FGV para esta aula

- Classe é molde; objeto é instância concreta.
- Encapsulamento não é sinônimo fraco de getter/setter; é controle de acesso e preservação de invariantes.
- Abstração não é apenas classe abstrata. Classe abstrata é mecanismo; abstração é princípio de modelagem.
- Herança deve representar 'é um tipo de'. Reaproveitamento de código, sozinho, não justifica herança.
- Composição é 'tem um/parte de' forte; herança é 'é um tipo de'.
- Sobrecarga muda lista de parâmetros; sobrescrita redefine método herdado compatível.

- Polimorfismo dinâmico depende do tipo real do objeto em tempo de execução.
- Java não tem herança múltipla de classes, mas permite implementar múltiplas interfaces.
- Atributo private da superclasse não é acessível diretamente pela subclasse.
- AOO procura classes e responsabilidades do domínio; não é desenho físico de banco nem estrutura de pacotes do framework.

## 14. Lista de questões autorais - padrão FGV

Instruções: resolva as 20 questões sem consultar o gabarito. As questões misturam conceito, caso prático, código, afirmativas, V/F, relacione, exceção/incorreta e leitura de situação concreta. As questões são autorais e usam provas reais da FGV apenas como referência de estilo, densidade e vocabulário.

1. Durante a análise de um sistema de gestão de documentos administrativos do TJSC, a equipe identifica que Documento, Unidade Judiciária, Assinatura Eletrônica e Usuário possuem identidade própria, atributos relevantes e comportamentos associados ao fluxo de tramitação. Já termos como criar, enviar, validar e publicar aparecem como ações descritas nos requisitos. Considerando a Análise Orientada a Objetos - AOO, a interpretação mais adequada é que:

(A) todos os substantivos devem ser convertidos em tabelas físicas, e todos os verbos devem ser descartados do modelo.

(B) os verbos destacados são candidatos naturais a classes principais, enquanto os substantivos representam apenas métodos.

(C) Documento, Unidade Judiciária, Assinatura Eletrônica e Usuário são candidatos a classes, desde que tenham relevância e responsabilidades no domínio; criar, enviar, validar e publicar tendem a indicar operações ou eventos a serem analisados.

(D) a AOO deve priorizar a definição de índices, chaves estrangeiras e tipos SQL antes de identificar classes de domínio.

(E) a existência de um termo no requisito é suficiente para obrigar sua implementação como classe concreta em Java.

2. Em uma aplicação Java, a classe ProcessoJudicial define atributos e comportamentos comuns a processos. Em tempo de execução, são criados dois objetos, p1 e p2, ambos da classe ProcessoJudicial, com o mesmo número processual informado por erro de integração, mas com referências distintas na memória. Do ponto de vista de orientação a objetos, é correto afirmar que:

(A) p1 e p2 são objetos distintos, ainda que possam estar em estado semelhante; a classe é o molde a partir do qual ambos foram criados.

(B) p1 e p2 são a mesma classe e, portanto, obrigatoriamente o mesmo objeto.

(C) p1 e p2 deixam de ser objetos porque possuem valores iguais em um atributo de negócio.

(D) ProcessoJudicial é uma instância concreta, enquanto p1 e p2 são classes derivadas.

(E) a identidade de um objeto OO depende exclusivamente do valor da chave primária no banco de dados.

3. Considere o trecho simplificado a seguir, usado em uma API interna de pagamentos:

```
public class OrdemPagamento {
    private BigDecimal valor;
    private Status status;

    public OrdemPagamento(BigDecimal valor) {
        if (valor.signum() <= 0) throw new IllegalArgumentException();
        this.valor = valor;
        this.status = Status.PENDENTE;
    }

    public void aprovar(Usuario usuario) {
        if (!usuario.temAlcada()) throw new AcessoNegadoException();
        this.status = Status.APROVADA;
    }
}
```

A característica de orientação a objetos mais diretamente evidenciada pelo uso de atributos privados e operações que preservam regras de validade do objeto é:

(A) herança múltipla, pois a classe combina regras de usuário e pagamento.

(B) polimorfismo paramétrico, pois a aprovação depende de um parâmetro.

(C) acoplamento temporal, pois o objeto depende da ordem dos métodos.

(D) encapsulamento, pois o estado interno é protegido e modificado por operações controladas.

(E) agregação, pois OrdemPagamento contém um objeto Usuario.

4. No modelo de um sistema processual, a equipe propôs que PeticaoInicial, Contestacao e Sentenca fossem especializações de DocumentoProcesso. Em outro ponto, propôs que DocumentoProcesso mantivesse uma referência para ArquivoDigital, responsável pelo conteúdo binário armazenado. As duas relações descritas são, respectivamente:

(A) composição e herança.

(B) generalização/especialização e associação/composição, conforme a força do vínculo de ciclo de vida entre DocumentoProcesso e ArquivoDigital.

(C) dependência e polimorfismo de sobrecarga.

(D) realização de interface e agregação fraca, necessariamente.

(E) sobrescrita e encapsulamento.

5. Em Java, uma classe Relatorio possui dois métodos gerar(): um sem parâmetros e outro que recebe um Periodo como parâmetro. Em uma subclasse RelatorioAuditoria, o método gerar(Periodo periodo) é redefinido com a mesma assinatura para aplicar regras específicas de auditoria. As situações descritas correspondem, respectivamente, a:

(A) abstração e composição.

(B) encapsulamento e agregação.

(C) herança múltipla e realização.

(D) dependência e associação.

(E) sobrecarga e sobrescrita de método.

6. A equipe pretende representar, em Java, diferentes documentos que podem ser assinados eletronicamente: Peticao, Sentenca e Despacho. Essas classes já possuem hierarquias próprias, mas todas devem expor o comportamento assinar(Usuario usuario). A solução conceitualmente mais adequada, sem forçar uma superclasse comum artificial, é:

(A) criar uma classe concreta Assinavel com todos os atributos de Peticao, Sentenca e Despacho.

(B) usar uma interface Assinavel, implementada pelas classes que oferecem o comportamento de assinatura.

(C) substituir todas as classes por métodos estáticos em uma classe UtilAssinatura.

(D) usar herança múltipla de classes, pois Java permite herdar de várias superclasses concretas.

(E) declarar os métodos como private, para que o polimorfismo ocorra por ocultação de dados.

7. Assinale a alternativa que apresenta a melhor distinção entre abstração e encapsulamento, no contexto de orientação a objetos.

(A) Abstração é a criação de chaves primárias; encapsulamento é a criação de chaves estrangeiras.

(B) Abstração é sempre implementada por herança múltipla; encapsulamento é sempre implementado por composição.

(C) Abstração seleciona aspectos essenciais de um conceito; encapsulamento controla o acesso ao estado interno e reúne dados e comportamentos relacionados.

(D) Abstração define a ordem de execução dos métodos; encapsulamento define a ordem dos pacotes no projeto.

(E) Abstração e encapsulamento são sinônimos, ambos significando apenas tornar atributos públicos.

8. Considere as afirmativas a seguir sobre orientação a objetos em Java.

I. Uma classe Java pode estender diretamente mais de uma classe concreta, desde que todas pertençam ao mesmo pacote.

II. Uma classe Java pode implementar mais de uma interface, permitindo que objetos suportem múltiplos contratos de comportamento.

III. Um atributo private declarado na superclasse pode ser acessado diretamente por uma subclasse, pois a herança elimina restrições de visibilidade.

Está correto o que se afirma em:

(A) I, apenas.

(B) II, apenas.

(C) III, apenas.

(D) I e II, apenas.

(E) II e III, apenas.

9. Relacione os conceitos de modelagem orientada a objetos às descrições.

1. Associação

2. Composição

3. Generalização

4. Realização

( ) Classe implementa contrato definido por interface.

( ) Relação todo-parte forte, com dependência relevante de ciclo de vida.

( ) Relação estrutural entre objetos/classes, sem necessariamente indicar herança.

( ) Relação entre supertipo e subtipo.

A sequência correta é:

(A) 4 - 2 - 1 - 3.

(B) 2 - 4 - 3 - 1.

(C) 1 - 3 - 4 - 2.

(D) 3 - 1 - 2 - 4.

(E) 4 - 1 - 2 - 3.

10. Em um sistema de protocolo eletrônico, uma classe ProtocoloController recebe requisições HTTP, valida regras complexas de negócio, calcula prazos, acessa diretamente o banco, gera PDF, envia e-mail e publica eventos de mensageria. Considerando coesão e acoplamento, a melhor avaliação é:

(A) a classe possui alta coesão, pois faz muitas tarefas importantes no mesmo fluxo.

(B) a classe demonstra polimorfismo por sobrecarga, pois lida com vários tipos de responsabilidade.

(C) a classe tende a apresentar baixa coesão e alto acoplamento, pois concentra responsabilidades heterogêneas e conhece muitos detalhes técnicos.

(D) a classe é obrigatoriamente uma entidade de domínio rica, pois tem muitos métodos públicos.

(E) a classe está correta por definição, pois todo controller deve conter regras de domínio e infraestrutura.

11. Na análise de um requisito, lê-se: 'O servidor protocola um requerimento, que é distribuído a uma unidade responsável. A unidade pode solicitar complementação e, ao final, emitir decisão administrativa.' Considerando uma leitura orientada a objetos, assinale a alternativa mais adequada.

(A) Servidor, Requerimento, Unidade e Decisão Administrativa são candidatos a classes ou conceitos de domínio; protocolar, solicitar complementação e emitir decisão indicam comportamentos, eventos ou operações a analisar.

(B) Protocolar, solicitar e emitir são as únicas classes de domínio, pois verbos sempre representam objetos persistentes.

- (C) Unidade deve ser um método de Servidor, pois aparece após a ação de distribuição.
- (D) Decisão Administrativa não pode ser classe, pois é resultado de um fluxo e não possui identidade possível.
- (E) Todos os termos devem ser transformados diretamente em componentes Angular.
12. Em um modelo de classes de análise, a classe Pedido possui uma coleção de ItemPedido; cada ItemPedido não faz sentido sem o respectivo Pedido e é criado/removido no ciclo de vida do Pedido. A relação mais adequada entre Pedido e ItemPedido é:
- (A) generalização, pois ItemPedido é um tipo de Pedido.
- (B) composição, pois há relação todo-parte forte com dependência de ciclo de vida.
- (C) realização, pois Pedido implementa ItemPedido.
- (D) dependência transitiva, pois os objetos pertencem a pacotes diferentes.
- (E) sobrecarga, pois Pedido possui vários itens.
13. Sobre análise orientada a objetos, assinale a alternativa INCORRETA.
- (A) A identificação de classes candidatas pode partir de substantivos dos requisitos, mas exige avaliação de relevância, identidade, responsabilidade e ciclo de vida.
- (B) Classes de fronteira tendem a representar interação com atores, telas, APIs ou sistemas externos.
- (C) Classes de controle podem coordenar fluxos de caso de uso ou aplicação.
- (D) Classes de entidade representam conceitos do domínio e podem possuir estado e regras relevantes.
- (E) Todo substantivo encontrado no requisito deve obrigatoriamente virar classe concreta persistida, sob pena de violar o paradigma OO.
14. Considere o pseudocódigo a seguir.

```
Documento doc = new Sentenca();  
doc.assinar(magistrado);
```

A classe Sentenca sobrescreve o método assinar() definido em Documento. A chamada doc.assinar(magistrado) executa a implementação de Sentenca. O fenômeno descrito está associado principalmente a:

- (A) sobrecarga estática de métodos por quantidade de parâmetros.
- (B) composição fraca entre Documento e Sentenca.
- (C) encapsulamento de atributo público.
- (D) polimorfismo por sobrescrita e despacho dinâmico.
- (E) normalização de modelo relacional.
15. Uma classe abstrata RelatorioProcessual define o método concreto formatarCabecalho() e declara o método abstrato carregarDados(). Subclasses como RelatorioMovimentacoes e RelatorioPartes implementam carregarDados(). Nesse caso, é correto afirmar que a classe abstrata:
- (A) não pode ser instanciada diretamente e pode combinar comportamento comum com operações abstratas a serem implementadas pelas subclasses.
- (B) funciona exatamente como uma enumeração, pois define uma lista fechada de valores.
- (C) representa apenas uma interface, pois classes abstratas não podem ter método concreto.
- (D) elimina a possibilidade de polimorfismo, pois subclasses não podem redefinir comportamentos.
- (E) obriga Java a usar herança múltipla de classes concretas.
16. Em uma revisão de modelagem, foi identificado que a classe DocumentoDigital expõe publicamente sua lista interna de assinaturas, permitindo que qualquer camada remova assinaturas sem validar permissões, ordem cronológica ou estado do documento. A crítica mais precisa, em termos de OO, é que:
- (A) há excesso de polimorfismo, pois listas não devem existir em objetos.

(B) há violação de encapsulamento, pois o estado interno pode ser alterado sem controle das invariantes do objeto.

(C) há uso indevido de herança múltipla, pois assinaturas pertencem a documentos.

(D) há baixa abstração porque o sistema utiliza classes em vez de funções globais.

(E) há composição, e composição é sempre proibida em modelos OO.

17. A equipe modelou Usuario como superclasse de Servidor, Magistrado e Advogado, apenas porque todos fazem login no sistema. Porém, Servidor, Magistrado e Advogado possuem naturezas institucionais distintas, regras diferentes e nem todo usuário autenticado representa necessariamente uma dessas categorias. A crítica mais adequada é:

(A) a herança está sempre correta quando classes compartilham qualquer atributo, como login.

(B) o uso de herança deve ser evitado porque Java não suporta classes.

(C) a modelagem provavelmente confundiu compartilhamento técnico de autenticação com relação conceitual 'é um tipo de'; composição, papéis ou associações podem representar melhor o domínio.

(D) a solução correta é transformar todos os usuários em métodos estáticos.

(E) a única alternativa seria usar banco NoSQL orientado a grafos.

18. Considere as afirmações sobre DTO - Data Transfer Object, entidade de domínio e classe de controle.

I. Um DTO pode ser uma classe, mas seu papel usual é transportar dados entre camadas ou sistemas, não necessariamente concentrar regra de negócio.

II. Uma entidade de domínio representa conceito relevante do problema e pode possuir comportamento associado às regras do domínio.

III. Uma classe de controle pode coordenar um caso de uso ou fluxo de aplicação, sem representar necessariamente uma entidade persistente.

Está correto o que se afirma em:

(A) I, apenas.

(B) I e II, apenas.

(C) II e III, apenas.

(D) I, II e III.

(E) III, apenas.

19. Em um modelo OO, a classe AssinaturaEletronica possui atributos dataHora, assinante, algoritmo e hashDocumento, além de operações validarIntegridade() e revogar(). O analista decide representá-la como atributo textual simples dentro de DocumentoProcesso, sem identidade nem comportamento. Considerando o cenário, a decisão é criticável porque:

(A) todo atributo textual deve virar herança múltipla.

(B) nenhum conceito relacionado a assinatura pode ser modelado em OO.

(C) objetos nunca podem ter atributos relacionados a segurança.

(D) um conceito com identidade, dados próprios, ciclo de vida e comportamentos relevantes tende a ser melhor representado como classe/conceito de domínio, não como simples texto.

(E) a assinatura deve ser sempre controller, independentemente do domínio.

20. Na modelagem de um sistema, a equipe criou uma interface Exportavel com o método exportar(Formato formato). As classes RelatorioProcessual, DocumentoDigital e HistoricoMovimentacao implementam essa interface. O benefício OO mais diretamente associado a essa decisão é:

(A) permitir que clientes tratem objetos diferentes por um contrato comum, favorecendo polimorfismo e reduzindo dependência de classes concretas.

(B) transformar automaticamente todas as classes em tabelas normalizadas.

(C) impedir que as classes tenham atributos privados.

(D) substituir a necessidade de testes de integração.

(E) garantir herança múltipla de implementação entre todas as classes.



## **15. Gabarito resumido**

1-C, 2-A, 3-D, 4-B, 5-E, 6-B, 7-C, 8-B, 9-A, 10-C, 11-A, 12-B, 13-E, 14-D, 15-A, 16-B, 17-C, 18-D, 19-D, 20-A

## 16. Comentários completos

Questão 1 - Gabarito: C

A correta é C. AOO procura conceitos relevantes do domínio e suas responsabilidades; substantivos com identidade, ciclo de vida e comportamento são candidatos a classes, enquanto verbos tendem a indicar operações, transições, eventos ou casos de uso. A está errada porque pula para tabela física e descarta verbos, quando eles também informam comportamento. B inverte a heurística: verbos não são naturalmente classes principais. D desloca a análise para banco físico, fora do foco. E é tentadora porque muitos materiais sugerem procurar substantivos, mas a heurística não é regra mecânica: relevância e responsabilidade importam.

Questão 2 - Gabarito: A

A correta é A. Classe é o molde; p1 e p2 são instâncias/objetos. Mesmo que atributos de negócio coincidam, podem ser objetos distintos. B confunde classe com objeto. C erra ao dizer que igualdade de valores elimina a natureza de objeto. D inverte classe e instância. E é tentadora para quem pensa em banco de dados, mas identidade OO não depende exclusivamente de chave primária; há identidade de referência e identidade conceitual, conforme o contexto.

Questão 3 - Gabarito: D

A correta é D. Os atributos privados e os métodos que validam transições preservam o estado interno e invariantes do objeto, caracterizando encapsulamento forte. A está errada porque não há herança múltipla. B usa termo de polimorfismo sem relação com o caso. C pode até existir em outros contextos, mas não é a característica central. E é tentadora porque há um parâmetro `Usuario`, mas isso não configura agregação estrutural como relação principal do modelo.

Questão 4 - Gabarito: B

A correta é B. `PeticaoInicial`, `Contestacao` e `Sentenca` como tipos de `DocumentoProcesso` representam generalização/especialização. `DocumentoProcesso` referenciar `ArquivoDigital` indica uma relação 'tem um', que pode ser associação, agregação ou composição conforme o ciclo de vida. A inverte a primeira relação. C e E misturam conceitos de método com modelagem estrutural. D é restritiva: não há necessariamente interface nem agregação fraca.

Questão 5 - Gabarito: E

A correta é E. Dois métodos `gerar()` com assinaturas diferentes na mesma classe caracterizam sobrecarga. Redefinir `gerar(Periodo)` na subclasse com assinatura compatível caracteriza sobrescrita. A, B, C e D usam conceitos reais, mas não explicam a distinção do enunciado. A pegadinha é confundir qualquer 'mesmo nome' com polimorfismo dinâmico: sobrecarga é resolvida pela assinatura, normalmente em compilação.

Questão 6 - Gabarito: B

A correta é B. Interface expressa contrato de comportamento para classes que não precisam compartilhar uma mesma superclasse artificial. A cria classe concreta indevida e mistura modelos. C empobrece OO e desloca comportamento para utilitário. D é falsa em Java: não há herança múltipla de classes concretas. E confunde visibilidade privada com polimorfismo; métodos `private` não formam contrato público para clientes.

Questão 7 - Gabarito: C

A correta é C. Abstração foca no essencial; encapsulamento protege estado e reúne dados/comportamentos. A traz conceitos de banco. B reduz abstração e encapsulamento a mecanismos específicos. D inventa relação com ordem de execução/pacotes. E é uma inversão grosseira: tornar atributos públicos tende a violar encapsulamento.

Questão 8 - Gabarito: B

A correta é B. I é falsa porque Java não permite que uma classe estenda diretamente múltiplas classes concretas. II é verdadeira: classes podem implementar múltiplas interfaces. III é falsa: atributo `private` da superclasse não é acessível diretamente pela subclasse; acesso deve ocorrer via métodos ou visibilidade adequada. A, C, D e E erram a composição das afirmativas.

Questão 9 - Gabarito: A

A correta é A: realização = classe implementa interface; composição = todo-parte forte; associação = vínculo estrutural; generalização = supertipo/subtipo. B, C, D e E embaralham termos que a FGV costuma aproximar. O ponto de prova é não confundir 'tem um' com 'é um tipo de' e não chamar implementação de interface de

herança de classe.

Questão 10 - Gabarito: C

A correta é C. Uma classe que valida domínio, acessa banco, gera PDF, envia e-mail e publica evento tende a concentrar responsabilidades heterogêneas, o que indica baixa coesão e alto acoplamento. A é tentadora porque a classe parece 'resolver o fluxo', mas isso não significa bom design. B usa polimorfismo indevidamente. D confunde muitos métodos com entidade de domínio. E transforma uma má prática comum em regra geral.

Questão 11 - Gabarito: A

A correta é A. Servidor, Requerimento, Unidade e Decisão Administrativa são candidatos plausíveis a classes/conceitos de domínio. Verbos indicam comportamentos, casos de uso, eventos ou métodos a analisar. B inverte substantivo/verbo. C transforma conceito institucional em método sem justificativa. D é falsa: resultado de fluxo pode ter identidade e ciclo de vida. E foge do nível de análise e vai para front-end.

Questão 12 - Gabarito: B

A correta é B. Pedido e ItemPedido formam relação todo-parte forte; o item não faz sentido fora do pedido e acompanha seu ciclo de vida. A confunde 'parte de' com 'tipo de'. C usa realização/interface sem base. D usa termo de dependência sem relação com o cenário. E confunde multiplicidade com sobrecarga de método.

Questão 13 - Gabarito: E

A incorreta é E. Nem todo substantivo vira classe concreta persistida; a análise deve avaliar relevância, identidade, responsabilidade e ciclo de vida. A, B, C e D são afirmações corretas: substantivos ajudam como candidatos, boundary/fronteira interage com atores/sistemas, control coordena fluxos, e entidade representa conceitos de domínio. A pegadinha é a heurística dos substantivos, que não pode virar regra absoluta.

Questão 14 - Gabarito: D

A correta é D. A variável é declarada como Documento, mas o objeto real é Sentença; a implementação executada é a de Sentença, por sobrescrita e despacho dinâmico. A descreve sobrecarga, que depende de assinaturas diferentes. B fala de composição sem base. C confunde encapsulamento com atributo público. E é assunto de banco de dados, não de OO.

Questão 15 - Gabarito: A

A correta é A. Classe abstrata não é instanciada diretamente e pode combinar métodos concretos com métodos abstratos. B confunde classe abstrata com enum. C é falsa porque classe abstrata pode ter método concreto. D erra: classes abstratas podem favorecer polimorfismo. E é falsa porque Java não exige nem permite herança múltipla de classes concretas.

Questão 16 - Gabarito: B

A correta é B. Expor lista interna permite alterações sem validação, quebrando encapsulamento e invariantes. A é falsa: listas podem existir em objetos. C não trata de herança múltipla. D não tem relação com preferência por classes ou funções. E é absurda tecnicamente: composição é permitida e frequentemente desejável; o problema é expor estado interno sem controle.

Questão 17 - Gabarito: C

A correta é C. Compartilhar login/autenticação é aspecto técnico; não prova relação conceitual 'é um tipo de'. Papéis, composição ou modelo de permissões podem representar melhor o domínio. A é a armadilha do reaproveitamento por herança. B é falsa porque Java suporta classes. D e E fogem do problema e não resolvem a semântica do domínio.

Questão 18 - Gabarito: D

A correta é D. I é verdadeira: DTO transporta dados e não necessariamente regra de negócio. II é verdadeira: entidade de domínio modela conceito relevante e pode ter comportamento. III é verdadeira: classe de controle coordena fluxo/caso de uso e não precisa ser persistente. As demais alternativas erram por excluir afirmativas corretas.

Questão 19 - Gabarito: D

A correta é D. AssinaturaEletronica tem dados, identidade, ciclo de vida e comportamentos relevantes; reduzi-la a texto pode empobrecer o modelo. A, B, C e E são distractores com exageros. A questão não diz que toda assinatura deve sempre ser classe, mas o cenário dá sinais fortes de que ela merece representação própria no domínio.

A correta é A. Interface permite que clientes operem sobre um contrato comum, tratando objetos diferentes de forma polimórfica e reduzindo dependência de classes concretas. B mistura OO com modelo relacional. C é falso: interfaces não impedem atributos privados nas classes. D foge para testes. E é falsa: implementar interface não é herança múltipla de implementação.

## 17. Checklist final do que memorizar

- Objeto = estado + comportamento + identidade.
- Classe = molde/tipo; objeto = instância concreta.
- Encapsulamento protege estado e preserva invariantes; não é só private + getter/setter.
- Abstração é seleção do essencial; classe abstrata é apenas um mecanismo possível.
- Herança/generalização = relação 'é um tipo de'. Composição/associação = relação 'tem um' ou 'usa'.
- Sobrecarga = mesmo nome, parâmetros diferentes. Sobrescrita = redefinição compatível em subclasse.
- Polimorfismo dinâmico: mesma mensagem, comportamentos específicos conforme o tipo real do objeto.
- Java: uma superclasse direta; múltiplas interfaces; private da superclasse não é acessado diretamente pela subclasse.
- AOO identifica classes, responsabilidades e colaborações; não é desenho físico de banco nem estrutura de framework.
- Alta coesão e baixo acoplamento são objetivos de boa modelagem OO.

## 18. Referências consultadas

- TJSC - Edital n. 10/2026, retificado em 10/04/2026 - disciplina Engenharia de Software e Desenvolvimento.
- Manual Mestre de Calibração FGV para Aulas e Questões - Projeto TJSC 2026.
- FGV Conhecimento - ALEGO 2025 - Analista Legislativo - Analista de Sistemas, prova objetiva, questões com OO, Java/OOP, SOLID e padrões.
- FGV Conhecimento - AL-AM 2025 - Analista Legislativo - Analista de Sistema, prova objetiva, questões com classe abstrata, encapsulamento, coesão e AOO.
- FGV Conhecimento - TJDFT 2022 - Analista Judiciário - Análise de Sistemas, prova objetiva, questão com código Java, herança, @Override e encapsulamento.
- Oracle/Dev.java - Objects, Classes, Interfaces, Packages, and Inheritance.
- Oracle Java Tutorials - Object-Oriented Programming Concepts.
- Java Language Specification SE 17 - classes, methods, overriding, overloading and interfaces.